

Déboguer son code

Lorsque vous développerez un programme, il est normal de rencontrer des erreurs. Ces erreurs peuvent être dues à une faute de frappe, à une mauvaise logique ou à une mauvaise utilisation des variables. Voici quelques conseils pour déboguer votre code JavaScript. Le navigateur web dispose d'outils de développement intégrés qui vous permettent de déboguer votre code JavaScript. Vous pouvez y accéder en appuyant sur **F12** ou en faisant un clic droit sur la page et en sélectionnant "Inspecter" ou "Outils de développement", en fonction du navigateur que vous utilisez.

NOTES TOUJOURS AVOIR LA CONSOLE OUVERTE LORSQUE VOUS DÉVELOPPEZ VOTRE CODE

Références

Les sites suivants sont de bonnes références pour apprendre à déboguer son code JavaScript. N'hésitez pas à les consulter si vous avez des questions ou si vous souhaitez approfondir vos connaissances.

- [MDN Web Docs - Documentation officielle de JavaScript](#)
- [W3Schools - Tutoriels et références JavaScript](#)
- [JavaScript.info - Tutoriel JavaScript moderne](#)

Commenter le code

En programmation, il est courant d'ajouter des commentaires dans le code pour expliquer ce que fait chaque partie du code. Les commentaires sont ignorés par le navigateur et ne sont pas exécutés. Ils servent uniquement à aider les développeurs à comprendre le code. Une bonne pratique est de commenter le code pour expliquer ce que fait chaque partie, surtout si le code est complexe ou si vous travaillez en équipe.

En JavaScript, vous pouvez ajouter un commentaire sur une seule ligne en utilisant `//` au début de la ligne, ou un commentaire multi-lignes en utilisant `/* ... */`.

```
// Ceci est un commentaire sur une seule ligne qui explique que la variable
nNombre contient un nombre
let nNombre = 5;
console.log(nNombre); // Ceci est un autre commentaire au bout de la ligne mais
plus difficile à lire. À garder pour les commentaires courts

/*
Ceci est un commentaire multi-lignes
qui peut s'étendre sur plusieurs lignes
et qui est utile pour expliquer des parties plus complexes du code
*/
let sMessage = "Bonjour";
console.log(sMessage); // Affiche: Bonjour
```

Si votre code ne fonctionne pas

Vous pouvez également utiliser des commentaires pour désactiver temporairement une partie de votre code. Cela peut être utile pour tester une partie du code sans l'exécuter ou pour laisser des notes dans le code. Si

plus rien ne va, vous pouvez commenter tout le code que vous avez écrit et le réactiver progressivement pour identifier la partie qui pose problème. Le raccourci dans Visual Studio Code pour commenter une ligne est **Ctrl + /** (Windows) ou **Cmd + /** (Mac). Pour commenter plusieurs lignes, sélectionnez les lignes et utilisez le même raccourci.

Utiliser la console pour afficher des messages et vous laisser des traces

Vous pouvez utiliser la fonction `console.log()` pour afficher des messages dans la console du navigateur. Cela vous permet de vérifier la valeur des variables à différents moments de l'exécution de votre code. Cette méthode sert au développement. Cela ne devrait pas être utilisé dans un programme final car cela peut ralentir l'exécution du code et polluer la console avec des messages inutiles. Cependant, c'est très utile pour le débogage.

Lorsque vous remettez votre code, vous pouvez supprimer les appels à `console.log()` ou les commenter en ajoutant `//` au début de la ligne. Cela permet de garder le code propre et lisible.

```
let sPrenom = "Alice";
let sNom = "Dupont";
console.log("Le prénom est: " + sPrenom); // Affiche: Le prénom est: Alice
console.log("Le nom est: " + sNom); // Affiche: Le nom est Dupont
```

Vérifier le type de valeur d'une variable

Vous pouvez utiliser `console.log()` pour afficher le type de valeur d'une variable en utilisant l'opérateur `typeof`. Cela peut être utile pour vérifier si une variable contient le type de valeur attendu, surtout lorsque vous travaillez avec des données saisies par l'utilisateur. Si jamais vous avez un doute sur le type d'une variable, vous pouvez utiliser `typeof` pour le vérifier.

```
let nNombre = 42;
console.log(typeof nNombre); // Affiche: number (car nNombre est un nombre)

let sTexte = "Bonjour";
console.log(typeof sTexte); // Affiche: string (car sTexte est une chaîne de caractères)
```

Demander une information à l'utilisateur

Vous pouvez utiliser la fonction `prompt()` pour demander une information à l'utilisateur. Cette fonction affiche une boîte de dialogue avec un champ de saisie et un bouton "OK". La valeur saisie par l'utilisateur est renvoyée sous forme de chaîne de caractères. Vous pouvez ensuite stocker cette valeur dans une variable pour l'utiliser dans votre programme.

`prompt()` sert au débogage et au développement. Cependant, vous apprendrez à créer vos propres boîtes de dialogue personnalisées pour interagir avec l'utilisateur. Vous ne devriez pas utiliser `prompt()` dans un programme final car cela peut être intrusif pour l'utilisateur et ne permet pas de personnaliser l'interface utilisateur.

Note: `prompt()` retourne toujours du texte, même si l'utilisateur saisit un nombre. Vous devrez peut-être convertir cette valeur en nombre si vous souhaitez effectuer des opérations mathématiques avec.

```
let sReponse = prompt("Quel est votre prénom ?"); // Affiche une boîte de dialogue
demandant le prénom
console.log("Bonjour, " + sReponse + "!"); // Affiche: Bonjour, Alice! (si
l'utilisateur a saisi "Alice")

let sAge = prompt("Quel est votre âge ?"); // Affiche une boîte de dialogue
demandant l'âge
let nAge = Number(sAge); // Convertit la réponse en nombre

if (nAge >= 18) {
    console.log("Vous êtes majeur."); // Affiche: Vous êtes majeur. (si
l'utilisateur a saisi un âge >= 18)
} else {
    console.log("Vous êtes mineur."); // Affiche: Vous êtes mineur. (si
l'utilisateur a saisi un âge < 18)
}
```

Utiliser les alertes pour afficher des messages

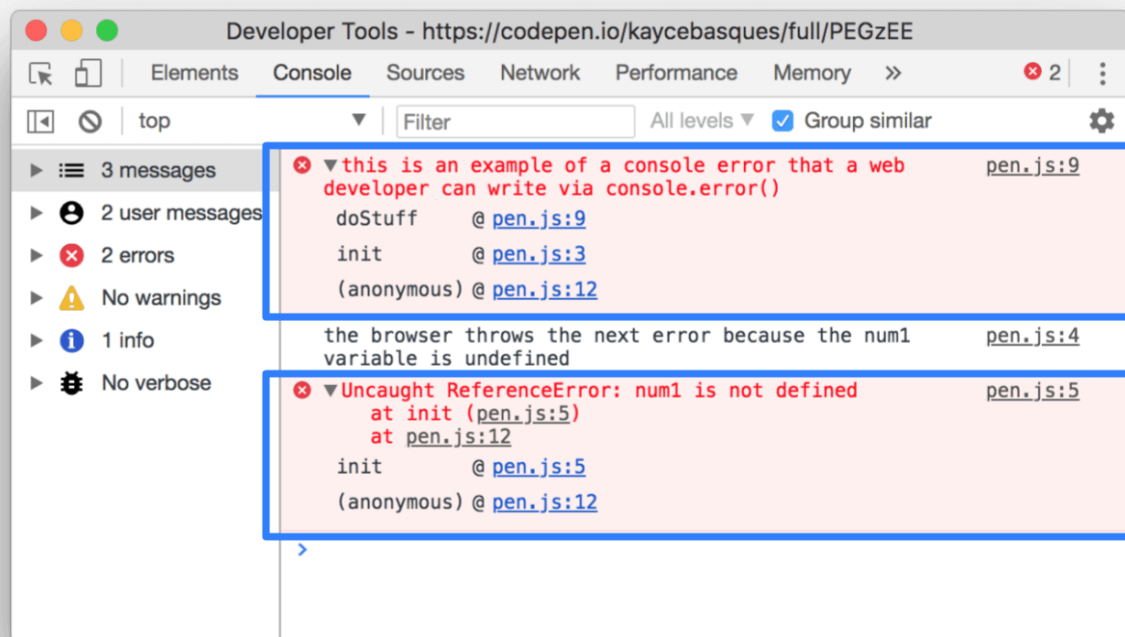
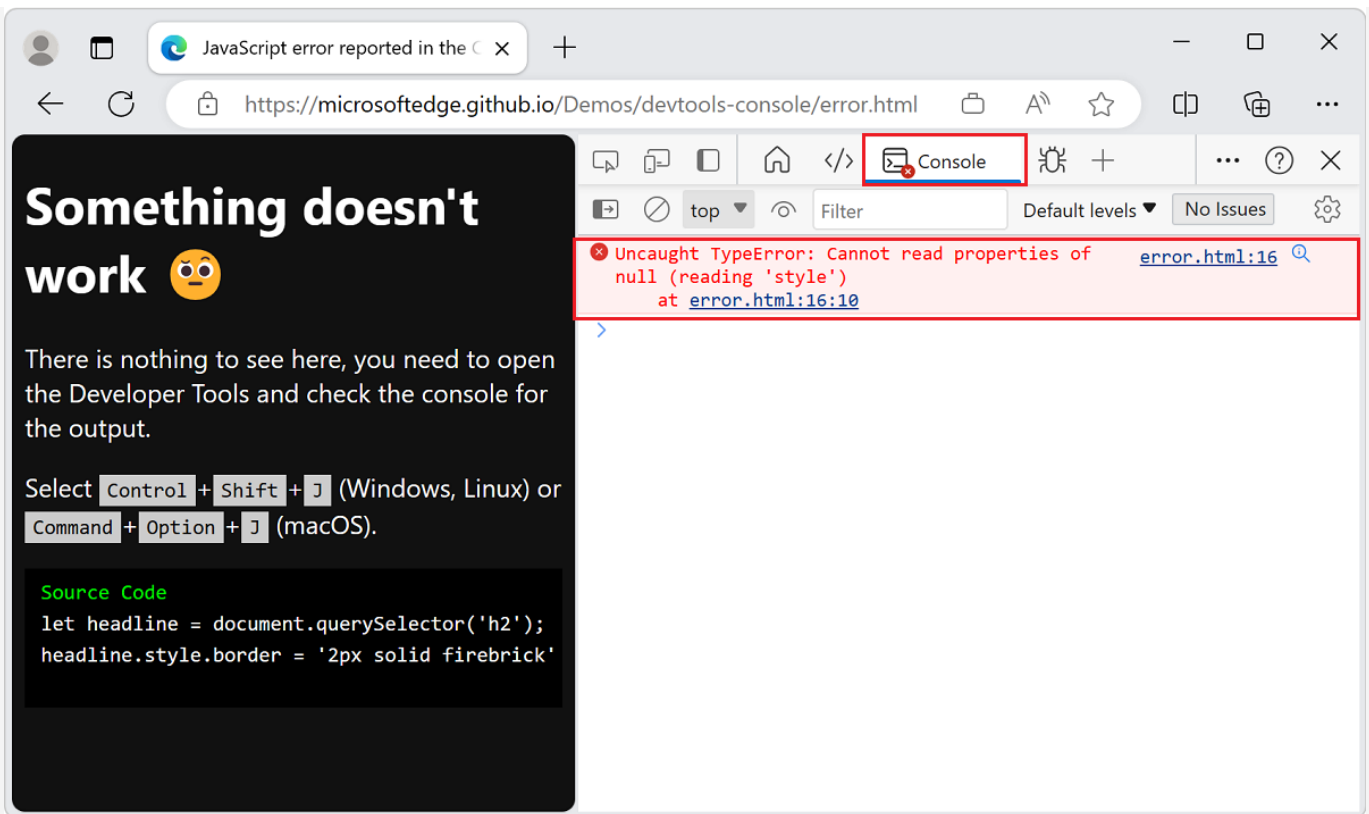
De la même manière que `console.log()`, vous pouvez utiliser la fonction `alert()` pour afficher un message à l'utilisateur dans une boîte de dialogue. Cela peut être utile pour afficher des messages d'erreur ou des informations importantes. Cependant, cela peut être intrusif car l'utilisateur doit cliquer sur "OK" pour fermer la boîte de dialogue. De plus, `alert()` bloque l'exécution du script jusqu'à ce que l'utilisateur ferme la boîte de dialogue, ce qui peut être gênant pour l'expérience utilisateur.

```
alert("Ceci est un message d'alerte !"); // Affiche une boîte de dialogue avec le
message
```

TOUJOURS AVOIR LA CONSOLE OUVERTE LORSQUE VOUS DÉVELOPPEZ VOTRE CODE

TOUJOURS AVOIR LA CONSOLE OUVERTE LORSQUE VOUS DÉVELOPPEZ VOTRE CODE. Cela vous permettra de voir les messages d'erreur et les messages que vous avez ajoutés avec `console.log()`.

Si vous rencontrez des erreurs dans votre code, ceux-ci seront affichés dans la console du navigateur. Regardez attentivement les messages d'erreur, car ils vous indiqueront souvent la ligne où l'erreur s'est produite et une description de l'erreur. Cela vous aidera à identifier et à corriger le problème ou à chercher de l'aide en ligne si nécessaire.



Voici quelques erreurs courantes :

- **SyntaxError**: Cela signifie qu'il y a une erreur de syntaxe dans votre code, comme une parenthèse non fermée.
- **ReferenceError**: Cela signifie que vous essayez d'utiliser une variable qui n'a pas été déclarée ou qui n'est pas accessible dans le contexte actuel.
- **TypeError**: Cela signifie que vous essayez d'effectuer une opération sur une valeur qui n'est pas du bon type, par exemple, essayer d'appeler une méthode sur `undefined` ou `null`.

